Here are the pseudocodes for each of the 50 algorithmic tasks:

1. Determine whether two given strings are anagrams of each other

```
FUNCTION ARE_ANAGRAMS(str1, str2)
  SORT str1
  SORT str2
  IF str1 == str2 THEN
    RETURN TRUE
  ELSE
    RETURN FALSE
  END IF
END FUNCTION
```

Example:

ARE_ANAGRAMS("listen", "silent")

returns

TRUE

2. Convert a non-negative decimal integer into its binary string representation

```
FUNCTION DECIMAL_TO_BINARY(n)
  IF n == 0 THEN
    RETURN "0"
  END IF
  binary = ""
  WHILE n > 0
    binary = MOD(n, 2) + binary
    n = FLOOR(n / 2)
  END WHILE
```

```
  RETURN binary
END FUNCTION
```

Example:

DECIMAL_TO_BINARY(10)

returns

"1010"

3. Determine whether a given year satisfies the leap year rules

```
FUNCTION IS_LEAP_YEAR(year)
  IF year MOD 4 == 0 AND (year MOD 100 != 0 OR year MOD 400 == 0) THEN
    RETURN TRUE
  ELSE
    RETURN FALSE
  END IF
END FUNCTION
```

Example:

IS_LEAP_YEAR(2020)

returns

TRUE

4. Find the intersection of two sorted arrays and return the common elements

```
FUNCTION INTERSECTION(arr1, arr2)
  i = 0
  j = 0
```

```
  intersection = []
  WHILE i < LENGTH(arr1) AND j < LENGTH(arr2)
    IF arr1[i] == arr2[j] THEN
      intersection += [arr1[i]]
      i += 1
      j += 1
    ELSE IF arr1[i] < arr2[j] THEN
      i += 1
    ELSE
      j += 1
    END IF
  END WHILE
  RETURN intersection
END FUNCTION
```

Example:

INTERSECTION([1, 2, 3, 4, 5], [2, 4, 6, 8, 10])

returns

[2, 4]

5. Remove punctuation from a sentence

```
FUNCTION REMOVE_PUNCTUATION(sentence)
  RETURN REPLACE_ALL(sentence, "[^a-zA-Z0-9\s]", "")
END FUNCTION
```

Example:

REMOVE_PUNCTUATION("Hello, world!")

returns

"Hello world"

6. Find the length of a given string

```
FUNCTION LENGTH(s)
  RETURN NUMBER_OF_CHARACTERS(s)
END FUNCTION
```

Example:

LENGTH("hello")

Returns 5

7. Split a sentence into individual words and store in a list

```
FUNCTION SPLIT_SENTENCE(sentence)
  RETURN SPLIT(sentence, " ")
END FUNCTION
```

Example:

SPLIT_SENTENCE("Hello world this is a test")

returns

["Hello", "world", "this", "is", "a", "test"]

8. Sort a list of words in alphabetical order

```
FUNCTION SORT_WORDS(words)
  RETURN SORT(words, ALPHABETICAL_ORDER)
```

```
END FUNCTION
```

Example:

```
SORT_WORDS(["hello", "world", "abc", "xyz"])
```

returns

```
["abc", "hello", "world", "xyz"]
```

9. Given an unsorted list of integers and a target value T, find the exact position of T

T in the list, or report that it doesn't exist

```
FUNCTION FIND_TARGET(arr, target)
  FOR i FROM 0 TO LENGTH(arr) - 1
    IF arr[i] == target THEN
      RETURN i
    END IF
  END FOR
  RETURN -1
END FUNCTION
```

Example:

```
FIND_TARGET([3, 1, 4, 2, 5], 2)
```

returns

3

10. Scan a paragraph of text and count how many times each individual vowel (a, e, i, o, u) appears

```
FUNCTION COUNT_VOWELS(text)
  vowels = ["a", "e", "i", "o", "u"]
```

```
 counts = []
 FOR v IN vowels
  count = 0
  FOR c IN text
   IF c == v THEN
    count += 1
   END IF
  END FOR
  counts += [count]
 END FOR
 RETURN counts
END FUNCTION
```

Example:

COUNT_VOWELS("The quick brown fox jumps over the lazy dog")

returns

[1, 3, 2, 4, 2]

11. Given a list of IP addresses that accessed a server today, remove all duplicates and return only the unique IP addresses

```
FUNCTION REMOVE_DUPLICATE_IPS(ips)
 RETURN UNIQUE(ips)
END FUNCTION
```

Example:

REMOVE_DUPLICATE_IPS(["192.168.1.1", "192.168.1.2", "192.168.1.1", "192.168.1.3"])

returns

["192.168.1.1", "192.168.1.2", "192.168.1.3"]

12. Given a square grid of numbers (N×N), calculate the sum of the numbers that lie on the main diagonal from top-left to bottom-right

```
FUNCTION SUM_DIAGONAL(grid)
 sum = 0
 FOR i FROM 0 TO LENGTH(grid) - 1
  sum += grid[i][i]
 END FOR
 RETURN sum
END FUNCTION
```

Example:

```
SUM_DIAGONAL([[1, 2, 3], [4, 5, 6], [7, 8, 9]])
```

returns

15

13. Reverse a string without using built-in reverse functions

```
FUNCTION REVERSE_STRING(s)
 reversed = ""
 FOR i FROM LENGTH(s) - 1 TO 0
  reversed += s[i]
 END FOR
 RETURN reversed
END FUNCTION
```

Example:

```
REVERSE_STRING("hello")
```

returns

"olleh"

14. Find the sum of all even numbers in a list

```
FUNCTION SUM_EVEN_NUMBERS(arr)
 sum = 0
 FOR num IN arr
  IF num MOD 2 == 0 THEN
   sum += num
  END IF
 END FOR
 RETURN sum
END FUNCTION
```

Example:

SUM_EVEN_NUMBERS([1, 2, 3, 4, 5])

returns

6

15. Find the second largest in an array

```
FUNCTION SECOND_LARGEST(arr)
 max = NEGATIVE_INFINITY
 second_max = NEGATIVE_INFINITY
 FOR num IN arr
  IF num > max THEN
   second_max = max
   max = num
```

```
    ELSE IF num > second_max AND num != max THEN
      second_max = num
    END IF
  END FOR
  RETURN second_max
END FUNCTION
```

Example:

SECOND_LARGEST([1, 2, 3, 4, 5])

returns

4

16. Find GCD of two numbers using Euclidean algorithm

```
FUNCTION GCD(a, b)
  WHILE b != 0
    temp = b
    b = a MOD b
    a = temp
  END WHILE
  RETURN a
END FUNCTION
```

Example:

GCD(48, 18)

returns

6

17. Compute the sum and average of a list of numbers

```
FUNCTION SUM_AND_AVERAGE(arr)
  sum = 0
  FOR num IN arr
    sum += num
  END FOR
  average = sum / LENGTH(arr)
  RETURN [sum, average]
END FUNCTION
```

Example:

```
SUM_AND_AVERAGE([1, 2, 3, 4, 5])
```

returns

```
[15, 3]
```

18. Count the frequency of each character in a string

```
FUNCTION COUNT_CHARACTERS(s)
  counts = {}
  FOR c IN s
    IF c IN counts THEN
      counts[c] += 1
    ELSE
      counts[c] = 1
    END IF
  END FOR
  RETURN counts
END FUNCTION
```

Example:

COUNT_CHARACTERS("hello")

returns

{'h': 1, 'e': 1, 'l': 2, 'o': 1}

19. Swap two variables without using a temporary third variable

```
FUNCTION SWAP(a, b)
 a = a + b
 b = a - b
 a = a - b
 RETURN [a, b]
END FUNCTION
```

Example:

SWAP(2, 3)

returns

[3, 2]

20. Determine all divisors of a given positive integer

```
FUNCTION FIND_DIVISORS(n)
 divisors = []
 FOR i FROM 1 TO n
  IF n MOD i == 0 THEN
   divisors += [i]
  END IF
```

```
  END FOR
  RETURN divisors
END FUNCTION
```

Example:

FIND_DIVISORS(12)

returns

[1, 2, 3, 4, 6, 12]

21. Create a process that determines whether a number is even or odd

```
FUNCTION IS_EVEN(n)
  IF n MOD 2 == 0 THEN
    RETURN TRUE
  ELSE
    RETURN FALSE
  END IF
END FUNCTION
``
```

continue from 22

assistant

Here are the pseudocodes for the remaining tasks:

22. Design a procedure that computes the factorial of a positive integer

```
FUNCTION FACTORIAL(n)
  IF n == 0 OR n == 1 THEN
    RETURN 1
  ELSE
    RETURN n * FACTORIAL(n - 1)
  END IF
END FUNCTION
```

Example:

FACTORIAL(5)

returns

120

23. Write a process that converts numerical scores into grade categories such as Pass, Merit, or Distinction

```
FUNCTION CONVERT_SCORE(score)
  IF score >= 90 THEN
    RETURN "Distinction"
  ELSE IF score >= 70 AND score < 90 THEN
    RETURN "Merit"
  ELSE IF score >= 50 AND score < 70 THEN
    RETURN "Pass"
  ELSE
    RETURN "Fail"
  END IF
END FUNCTION
```

Example:

CONVERT_SCORE(85)

returns

"Merit"

24. Create a process that checks whether a password contains at least 8 characters, one number, and one uppercase letter

```
FUNCTION CHECK_PASSWORD(password)
 IF LENGTH(password) < 8 THEN
  RETURN FALSE
 END IF
 has_number = FALSE
 has_uppercase = FALSE
 FOR c IN password
  IF c IS A NUMBER THEN
   has_number = TRUE
  ELSE IF c IS AN UPPERCASE LETTER THEN
   has_uppercase = TRUE
  END IF
 END FOR
 IF has_number AND has_uppercase THEN
  RETURN TRUE
 ELSE
  RETURN FALSE
 END IF
END FUNCTION
```

Example:

CHECK_PASSWORD("Password123")

returns

TRUE

25. Find the length of the longest substring without repeating characters in a given string

```
FUNCTION LONGEST_SUBSTRING(s)
  max_length = 0
  start = 0
  char_index = {}
  FOR i FROM 0 TO LENGTH(s) - 1
    IF s[i] IN char_index AND char_index[s[i]] >= start THEN
      start = char_index[s[i]] + 1
    END IF
    char_index[s[i]] = i
    max_length = MAX(max_length, i - start + 1)
  END FOR
  RETURN max_length
END FUNCTION
```

Example:

LONGEST_SUBSTRING("abcabcbb")

returns

3

26. Perform level-order traversal on a binary tree and return node values grouped by depth

```
FUNCTION LEVEL_ORDER_TRAVERSAL(root)
  result = []
  queue = [root]
  WHILE queue IS NOT EMPTY
```

```
    level = []
    FOR i FROM 0 TO LENGTH(queue) - 1
      node = DEQUEUE(queue)
      level += [node.value]
      IF node.left IS NOT NULL THEN
        ENQUEUE(queue, node.left)
      END IF
      IF node.right IS NOT NULL THEN
        ENQUEUE(queue, node.right)
      END IF
    END FOR
    result += [level]
  END WHILE
  RETURN result
END FUNCTION
```

Example:

```
LEVEL_ORDER_TRAVERSAL(root)
```

returns

```
[[1], [2, 3], [4, 5, 6, 7]]
```

27. Rotate a square 2D matrix 90 degrees clockwise in-place

```
FUNCTION ROTATE_MATRIX(matrix)
  n = LENGTH(matrix)
  FOR layer FROM 0 TO n / 2
    first = layer
    last = n - layer - 1
    FOR i FROM first TO last
```

```
    top = matrix[layer][i]
    matrix[layer][i] = matrix[last - i + first][first]
    matrix[last - i + first][first] = matrix[last][last - i + first]
    matrix[last][last - i + first] = matrix[i][last]
    matrix[i][last] = top
  END FOR
 END FOR
 RETURN matrix
END FUNCTION
```

Example:

ROTATE_MATRIX([[1, 2, 3], [4, 5, 6], [7, 8, 9]])

returns

[[7, 4, 1], [8, 5, 2], [9, 6, 3]]

28. For a fixed window size sliding across an array, return the maximum value within each window position

```
FUNCTION MAX_IN_WINDOW(arr, window_size)
 result = []
 FOR i FROM 0 TO LENGTH(arr) - window_size
  max_value = NEGATIVE_INFINITY
  FOR j FROM i TO i + window_size - 1
   IF arr[j] > max_value THEN
    max_value = arr[j]
   END IF
  END FOR
  result += [max_value]
 END FOR
 RETURN result
```

```
END FUNCTION
```

Example:

```
MAX_IN_WINDOW([1, 2, 3, 4, 5], 3)
```

returns

```
[3, 4, 5]
```

29. Build a text classification model to categorize text into predefined categories

```
FUNCTION TEXT_CLASSIFICATION(text, categories)
  # Train a machine learning model using the given categories
  model = TRAIN_MODEL(categories)
  # Use the trained model to classify the given text
  RETURN CLASSIFY_TEXT(text, model)
END FUNCTION
```

Example:

```
TEXT_CLASSIFICATION("This is a sports article", ["sports", "politics", "entertainment"])
```

returns

```
"sports"
```

30. Create a topic modelling system to extract underlying topics from a large corpus

```
FUNCTION TOPIC_MODELLING(corpus)
  # Use a topic modelling algorithm such as Latent Dirichlet Allocation (LDA) to extract topics
  topics = EXTRACT_TOPICS(corpus)
  RETURN topics
END FUNCTION
```

Example:

TOPIC_MODELLING(["This is a sports article", "This is a politics article"])

returns

["sports", "politics"]

31. Develop a co-reference resolution system to identify relationships between pronouns and nouns

```
FUNCTION CO_REFERENCE_RESOLUTION(text)
  # Use a co-reference resolution algorithm to identify relationships
  relationships = IDENTIFY_RELATIONSHIPS(text)
  RETURN relationships
END FUNCTION
```

Example:

CO_REFERENCE_RESOLUTION("John said he would go to the store")

returns

{"he": "John"}

32. Build a question answering system to answer user queries based on a given text

```
FUNCTION QUESTION_ANSWERING(text, query)
  # Use a question answering algorithm to find the answer
  answer = FIND_ANSWER(text, query)
  RETURN answer
END FUNCTION
```

Example:

QUESTION_ANSWERING("The capital of France is Paris", "What is the capital of France")

returns

"Paris"

33. Implement a method to find the exact square root of a non-negative integer using binary search, rounded down to the nearest integer, without using built-in math functions

```
FUNCTION SQUARE_ROOT(n)
 IF n < 0 THEN
  RETURN ERROR
 END IF
 IF n == 0 OR n == 1 THEN
  RETURN n
 END IF
 low = 0
 high = n
 WHILE low <= high
  mid = (low + high) / 2
  IF mid * mid <= n THEN
   low = mid + 1
  ELSE
   high = mid - 1
  END IF
 END WHILE
 RETURN high
END FUNCTION
```

Example:

SQUARE_ROOT(16)

returns

4

34. Find all elements that appear in two separate unsorted lists, ensuring the result contains no duplicates

```
FUNCTION COMMON_ELEMENTS(list1, list2)
  RETURN UNIQUE_INTERSECTION(list1, list2)
END FUNCTION
```

Example:

COMMON_ELEMENTS([1, 2, 3], [2, 3, 4])

returns

[2, 3]

35. Design a memory buffer that holds a fixed number of items. When it runs out of space, it must automatically discard the item that hasn't been accessed for the longest time

```
FUNCTION MEMORY_BUFFER(capacity)
  buffer = []
  timestamps = {}
  FOR item IN ITEMS
    IF LENGTH(buffer) >= capacity THEN
      oldest_item = FIND_OLDEST_ITEM(buffer, timestamps)
      REMOVE_ITEM(buffer, oldest_item)
    END IF
    ADD_ITEM(buffer, item)
    UPDATE_TIMESTAMP(timestamps, item)
  END FOR
  RETURN buffer
END FUNCTION
```

Example:

MEMORY_BUFFER(3)

returns

[item1, item2, item3]

36. An ordered list of numbers was rotated at an unknown pivot point (e.g., [4,5,6,1,2,3]). Find the index of a target element in this list in logarithmic time

```
FUNCTION FIND_TARGET(rotated_list, target)
 low = 0
 high = LENGTH(rotated_list) - 1
 WHILE low <= high
  mid = (low + high) / 2
  IF rotated_list[mid] == target THEN
   RETURN mid
  ELSE IF rotated_list[low] <= rotated_list[mid] THEN
   IF target >= rotated_list[low] AND target < rotated_list[mid] THEN
    high = mid - 1
   ELSE
    low = mid + 1
   END IF
  ELSE
   IF target > rotated_list[mid] AND target <= rotated_list[high] THEN
    low = mid + 1
   ELSE
    high = mid - 1
   END IF
  END IF
 END WHILE
```

```
  RETURN -1
END FUNCTION
```

Example: `FIND

continue from 37

assistant

Here are the pseudocodes for the remaining tasks:

37. Implement a stack using an array (push/pop/peek)

```
CLASS STACK
  Constructor()
    array = []
    top = -1

  FUNCTION PUSH(element)
    top += 1
    array[top] = element

  FUNCTION POP()
    IF top == -1 THEN
      RETURN NULL
    END IF
    element = array[top]
    top -= 1
    RETURN element
```

```
 FUNCTION PEEK()
  IF top == -1 THEN
   RETURN NULL
  END IF
  RETURN array[top]

END CLASS
```

Example:

```
stack = new STACK(); stack.PUSH(1); stack.PUSH(2); stack.POP();
```

returns

2

38. Find the first non-repeating character in a string

```
FUNCTION FIRST_NON_REPEATING_CHARACTER(s)
 char_count = {}
 FOR c IN s
  IF c IN char_count THEN
   char_count[c] += 1
  ELSE
   char_count[c] = 1
  END IF
 END FOR
 FOR c IN s
  IF char_count[c] == 1 THEN
   RETURN c
  END IF
```

```
  END FOR
  RETURN NULL
END FUNCTION
```

Example:

```
FIRST_NON_REPEATING_CHARACTER("aabbc")
```

returns

c

39. Detect cycle in a singly linked list (Floyd's cycle detection)

```
FUNCTION HAS_CYCLE(head)
  tortoise = head
  hare = head
  WHILE tortoise != NULL AND hare != NULL AND hare.next != NULL
    tortoise = tortoise.next
    hare = hare.next.next
    IF tortoise == hare THEN
      RETURN TRUE
    END IF
  END WHILE
  RETURN FALSE
END FUNCTION
```

Example:

```
HAS_CYCLE(node1)
```

returns

TRUE

if there is a cycle,

FALSE

otherwise

40. Implement a queue using two stacks

```
CLASS QUEUE
 stack1 = []
 stack2 = []

 FUNCTION ENQUEUE(element)
  stack1.push(element)

 FUNCTION DEQUEUE()
  IF stack2 IS EMPTY THEN
   WHILE stack1 IS NOT EMPTY
    stack2.push(stack1.pop())
   END WHILE
  END IF
  RETURN stack2.pop()

END CLASS
```

Example:

queue = new QUEUE(); queue.ENQUEUE(1); queue.ENQUEUE(2); queue.DEQUEUE();

returns

1

41. Solve the N Queens Problem for N arbitrary board size N

```
FUNCTION N_QUEENS(N)
 board = []
 FOR i FROM 0 TO N-1
  board[i] = []
  FOR j FROM 0 TO N-1
   board[i][j] = 0
  END FOR
 END FOR
 FUNCTION IS_SAFE(row, col)
  FOR i FROM 0 TO row-1
   IF board[i][col] == 1 THEN
    RETURN FALSE
   END IF
   IF col - (row - i) >= 0 AND board[i][col - (row - i)] == 1 THEN
    RETURN FALSE
   END IF
   IF col + (row - i) < N AND board[i][col + (row - i)] == 1 THEN
    RETURN FALSE
   END IF
  END FOR
  RETURN TRUE
 END FUNCTION
 FUNCTION PLACE_QUEENS(row)
  IF row == N THEN
   RETURN TRUE
  END IF
  FOR col FROM 0 TO N-1
   IF IS_SAFE(row, col) THEN
```

```
      board[row][col] = 1
      IF PLACE_QUEENS(row + 1) THEN
        RETURN TRUE
      END IF
      board[row][col] = 0
    END IF
  END FOR
  RETURN FALSE
 END FUNCTION
 IF PLACE_QUEENS(0) THEN
  RETURN board
 ELSE
  RETURN NULL
 END IF
END FUNCTION
```

Example:

N_QUEENS(4)

returns a 4x4 board with 4 queens placed safely

42. Find all pairs in an array that sum to a given target value

```
FUNCTION FIND_PAIRS(arr, target)
 pairs = []
 FOR i FROM 0 TO LENGTH(arr) - 1
  FOR j FROM i + 1 TO LENGTH(arr) - 1
   IF arr[i] + arr[j] == target THEN
    pairs += [(arr[i], arr[j])]
   END IF
  END FOR
```

```
  END FOR
  RETURN pairs
END FUNCTION
```

Example:

```
FIND_PAIRS([1, 2, 3, 4, 5], 7)
```

returns

```
[(2, 5), (3, 4)]
```

43. Solve the Tower of Hanoi problem for N discs

```
FUNCTION TOWER_OF_HANOI(N)
  FUNCTION MOVE(n, from_rod, to_rod, aux_rod)
    IF n == 1 THEN
      PRINT("Move disk 1 from rod " + from_rod + " to rod " + to_rod)
    ELSE
      MOVE(n - 1, from_rod, aux_rod, to_rod)
      PRINT("Move disk " + n + " from rod " + from_rod + " to rod " + to_rod)
      MOVE(n - 1, aux_rod, to_rod, from_rod)
    END IF
  END FUNCTION
  MOVE(N, "A", "C", "B")
END FUNCTION
```

Example:

```
TOWER_OF_HANOI(3)
```

prints the sequence of moves to solve the puzzle

44. Find the contiguous subarray with the maximum sum (Kadane's algorithm)

```
FUNCTION MAX_SUBARRAY_SUM(arr)
  max_sum = NEGATIVE_INFINITY
  current_sum = 0
  FOR num IN arr
    current_sum = MAX(num, current_sum + num)
    max_sum = MAX(max_sum, current_sum)
  END FOR
  RETURN max_sum
END FUNCTION
```

Example:

```
MAX_SUBARRAY_SUM([-2, 1, -3, 4, -1, 2, 1, -5, 4])
```

returns

6

45. Create instructions for adding two matrices of equal size

```
FUNCTION ADD_MATRICES(matrix1, matrix2)
  result = []
  FOR i FROM 0 TO LENGTH(matrix1) - 1
    row = []
    FOR j FROM 0 TO LENGTH(matrix1[0]) - 1
      row += [matrix1[i][j] + matrix2[i][j]]
    END FOR
    result += [row]
  END FOR
  RETURN result
END FUNCTION
```

Example:

ADD_MATRICES([[1, 2], [3, 4]], [[5, 6], [7, 8]])

returns

[[6, 8], [10, 12]]

46. Develop a process for multiplying two matrices

```
FUNCTION MULTIPLY_MATRICES(matrix1, matrix2)
 result = []
 FOR i FROM 0 TO LENGTH(matrix1) - 1
  row = []
  FOR j FROM 0 TO LENGTH(matrix2[0]) - 1
   sum = 0
   FOR k FROM 0 TO LENGTH(matrix1[0]) - 1
    sum += matrix1[i][k] * matrix2[k][j]
   END FOR
   row += [sum]
  END FOR
  result += [row]
 END FOR
 RETURN result
END FUNCTION
```

Example:

MULTIPLY_MATRICES([[1, 2], [3, 4]], [[5, 6], [7, 8]])

returns

[[19, 22], [43, 50]]

47. Create an algorithm that updates inventory quantities after purchases and deliveries

```
FUNCTION UPDATE_INVENTORY(purchases, deliveries)
 inventory = []
 FOR item IN purchases
  IF item IN inventory THEN
   inventory[item] -= purchases[item]
  ELSE
   inventory[item] = -purchases[item]
  END IF
 END FOR
 FOR item IN deliveries
  IF item IN inventory THEN
   inventory[item] += deliveries[item]
  ELSE
   inventory[item] = deliveries[item]
  END IF
 END FOR
 RETURN inventory
END FUNCTION
```

Example:

UPDATE_INVENTORY({"item1": 10, "item2": 20}, {"item1": 5, "item3": 15})

returns

{"item1": -5, "item2": -20, "item3": 15}

48. Develop a recursive process that searches through nested folders to locate a specific file type

```
FUNCTION FIND_FILE(directory, file_type)
  files = []
  FOR item IN directory
    IF item IS A FILE AND item.endswith(file_type) THEN
      files += [item]
    ELSE IF item IS A DIRECTORY THEN
      files += FIND_FILE(item, file_type)
    END IF
  END FOR
  RETURN files
END FUNCTION
```

Example:

```
FIND_FILE("/path/to/directory", ".txt")
```

returns a list of all

.txt

files in the directory and its subdirectories

49. Create a pathfinding solution that guides a user through a maze using graph traversal

```
FUNCTION PATHFINDING(maze, start, end)
  graph = []
  FOR cell IN maze
    IF cell IS NOT A WALL THEN
      graph += [cell]
    END IF
  END FOR
```

continue from 50

assistant

Here are the pseudocodes for the remaining tasks:

50. Develop algorithms enabling autonomous drones to coordinate movement while avoiding collisions

```
FUNCTION COORDINATE_DRONES(drones, obstacles)
  FOR drone IN drones
    IF drone IS NEAR OBSTACLE THEN
      ADJUST_TRAJECTORY(drone, obstacles)
    END IF
    IF drone IS NEAR OTHER_DRONE THEN
      ADJUST_TRAJECTORY(drone, other_drone)
    END IF
  END FOR
  FOR obstacle IN obstacles
    IF obstacle IS NEAR DRONE THEN
      ADJUST_TRAJECTORY(obstacle, drone)
    END IF
  END FOR
  RETURN TRAJECTORIES
END FUNCTION
```

Example:

```
COORDINATE_DRONES([drone1, drone2, drone3], [obstacle1, obstacle2])
```

returns a list of adjusted trajectories for each drone to avoid collisions with obstacles and other drones.

Please note that this pseudocode is a simplified representation of the problem and actual implementation would require more complex algorithms and techniques, such as:

Sensor data processing and integration

Real-time trajectory planning and control

Collision avoidance and detection

Machine learning and decision-making